1 介绍

常说，“性能为王”。十年前是真的，现在当然也是如此。根据 [domo.com, 2017] 的数据，在 2017 年，全球每天创造了 2.5 个万亿[1]字节的数据，如 [statista.com, 2018] 所预测的那样，这个数字每年增长 25%。在我们日益数据中心化的世界中，信息交换的增长推动了对更快软件（SW）和更快硬件（HW）的需求。可以说，数据增长不仅对计算能力提出了需求，还对存储和网络系统提出了需求。

在个人电脑时代[2]，开发人员通常直接在操作系统之上进行编程，可能之间会有一些库。随着世界进入云时代，软件堆栈变得更加深奥和复杂。开发人员大多数时间都在工作的顶层堆栈已经远离了硬件。这些额外的层次抽象出实际的硬件，从而允许使用新型加速器来处理新兴工作负载。然而，这种演进的负面影响是，现代应用程序的开发人员对其软件运行的实际硬件的了解较少。

多亏了摩尔定律，软件程序员几十年来一直“轻松愉快”。过去，一些软件供应商更倾向于等待新一代硬件以加速其应用程序，而不是花费人力资源改进其代码。从图 1 可以看出，单线程性能增长正在放缓。单线程性能是指在隔离环境中测量时 CPU 核心内的单个硬件线程的性能。

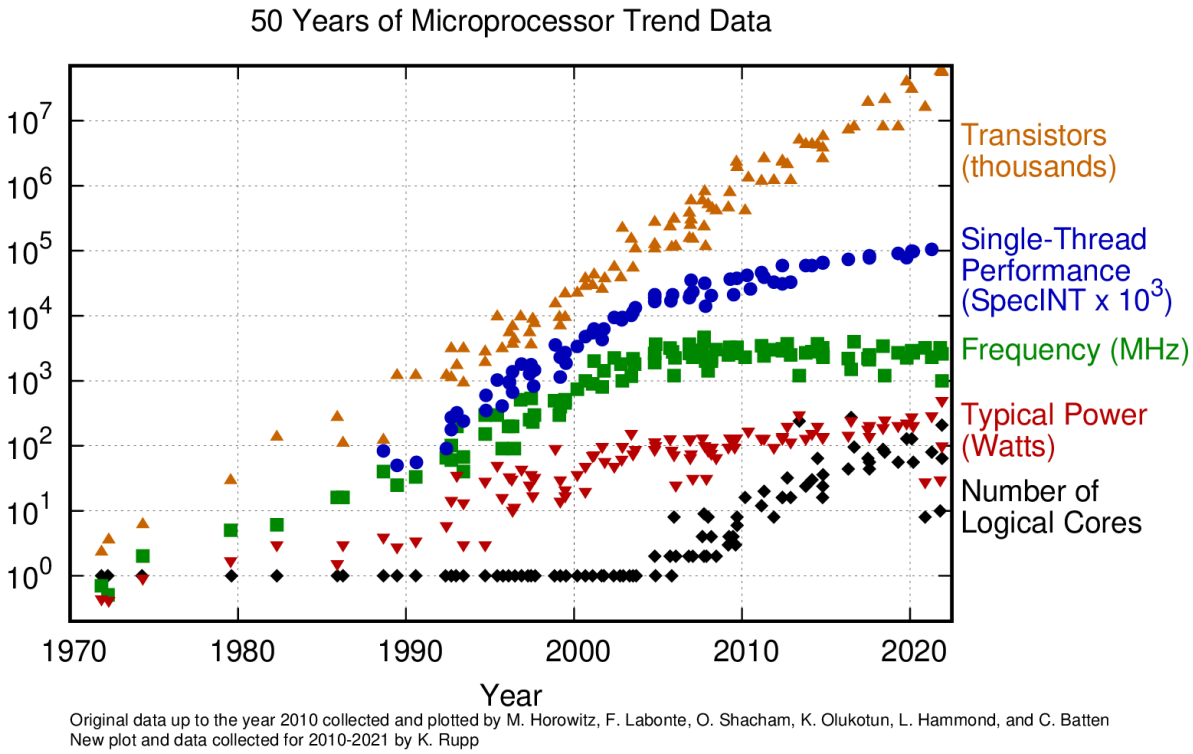


Figure 1: 50 Years of Microprocessor Trend Data. © Image by K. Rupp via karlrupp.net

当不再每个硬件世代都能提供显著的性能提升 [Leiserson et al., 2020] 时，我们必须开始更加关注代码运行的速度。在寻求提高性能的方法时，开发人员不应依赖硬件。相反，他们应该开始优化其应用程序的代码。

> “当今软件非常低效；现在是软件程序员再次变得擅长优化的黄金时代。”- 美国企业家和投资者马克·安德森（Marc Andreessen），摘自 a16z Podcast，2020 年

[1] 万亿是千的六次方（10^{18}）。

[2] 从 1990 年代末到 2000 年代末，个人电脑占据了计算设备市场的主导地位。

Personal Experience: 在英特尔工作期间，我时常听到同样的故事：当英特尔的客户在其应用程序中遇到慢速时，他们会立即无意识地开始责怪英特尔的 CPU 速度慢。但是当英特尔派遣我们的性能高手与他们合作并帮助他们改进应用程序时，往往会将其加速 2 倍，有时甚至是 10 倍。

达到高水平的性能具有挑战性，并且通常需要大量的努力，但希望这本书能给你提供帮助的工具。

1.1 为什么我们仍然需要性能调优?

现代 CPU 每年都在增加越来越多的核心。截至 2019 年底，您可以购买到一款高端服务器处理器，其逻辑核心数量超过 100 个。这非常令人印象深刻，但这并不意味着我们不再需要关注性能。很多时候，随着 CPU 核心数量的增加，应用程序的性能可能并不会提升。典型的通用多线程应用程序的性能并不总是随着我们分配给任务的 CPU 核心数量线性增长的。了解为什么会发生这种情况以及可能的解决方法对于产品未来的增长至关重要。不能进行适当的性能分析和调优会导致性能和金钱的浪费，并可能毁掉产品。

根据 [Leiserson et al., 2020]，至少在短期内，大多数应用程序的性能收益的很大一部分将来自软件堆栈。可悲的是，应用程序默认情况下并不会获得最佳性能。该论文还提供了一个很好的例子，说明了在源代码级别上可以进行的性能改进潜力。在表1中总结了对两个 4096×4096 矩阵相乘的程序进行性能工程的加速效果。通过应用多个优化，最终得到的程序运行速度提高了 60000 多倍。提供这个例子的原因不是挑剔 Python 或 Java（它们是很好的语言），而是打破了软件默认具有“足够好”的性能的观念。

Table 1: 在具有总计 60GB 内存的双插槽 Intel Xeon E5-2666 v3 系统上运行的一个程序，该程序执行两个 4096×4096 矩阵相乘的加速效果。来源：[Leiserson et al., 2020]。

版本	实现	绝对加速	相对加速
1	Python	1	—
2	Java	11	10.8
3	C	47	4.4
4	并行循环	366	7.8
5	并行分治	6,727	18.4
6	加向量化	23,224	3.5
7	加 AVX 指令集	62,806	2.7

以下是阻止系统默认达到最佳性能的一些最重要因素：

1. **CPU 限制**：很容易问：“为什么硬件不能解决我们所有的问题？”现代 CPU 以令人难以置信的速度执行指令，并且每一代都在变得更好。但是，如果用于执行工作的指令不是最佳的，甚至是多余的，它们就无法做太多事情。处理器不能通过魔法将次优代码转换为性能更好的代码。例如，如果我们使用 BubbleSort 算法实现排序例程，CPU 将不会尝试识别并使用更好的替代方案，例如 QuickSort。它会盲目地执行被告知要执行的任何操作。
2. **编译器限制**：“但是编译器不是应该做这些吗？为什么编译器不能解决我们所有的问题？”的确，现在的编译器非常智能，但仍然可能生成次优代码。编译器擅长消除冗余工作，但是当涉及到更复杂的决策，如函数内联、循环展开等时，它们可能不会生成最佳的代码。例如，是否应该将一个函数始终内联到调用它的地方，这个问题没有二元的“是”或“否”答案。这通常取决于编译器应该考虑的许多因素。通常，编译器依赖于复杂的成本模型和启发式算法，这些算法可能不适用于每种可能的情况。此外，除非编译器确定这样做是安全的，并且不会影响生成的机器代码的正确性，否则它们不能执行优化。对于编译器开发人员来说，确保特定优化在所有可能的情况下生成正确的代码可能非常困难，因此他们通常必须保守行事，并避免进行一些优化。最后，编译器通常不会转换程序使用的数据结构，这在性能方面也是至关重要的。

3. **算法复杂度分析限制**：开发人员经常过度关注算法的复杂度分析，这导致他们选择具有最优算法复杂度的流行算法，即使它对于给定问题可能不是最有效的。考虑两种排序算法，插入排序和快速排序，后者在平均情况的大 O 符号中显然更胜一筹：插入排序是 O(N^2)，而快速排序仅为 O(N log N)。然而，对于相对较小的N值（最多 50 个元素），插入排序的性能优于快速排序。复杂度分析无法考虑各种算法的分支预测和缓存效果，因此人们只是将它们封装在隐含的常数C中，有时这可能会对性能产生重大影响。盲目地信任大 O 符号而没有在目标工作负载上进行测试可能会使开发人员走上错误的道路。因此，对于某个特定问题来说，最知名的算法并不一定是实践中最高效的。

以上所述的限制留下了调优我们的软件以发挥其全部潜力的余地。广义上来说，软件堆栈包括许多层，例如固件、BIOS、操作系统、库以及应用程序的源代码。但由于大多数较低的软件层不在我们的直接控制之下，因此重点将放在源代码上。我们将经常涉及的另一个重要的软件是编译器。通过让编译器生成所需的机器代码，可以获得令人满意的加速效果，通过各种提示方法。您将在本书中找到许多这样的例子。

> **Personal Experience:** 要成功地在您的应用程序中实现所需的改进，您不必成为编译器专家。根据我的经验，至少 90% 的所有转换都可以在源代码级别完成，而无需深入研究编译器源代码。尽管如此，了解编译器的工作原理以及如何使其按照您的意愿进行操作在与性能相关的工作中始终是有利的。

此外，如今，使应用程序能够通过将其分布在许多核心上扩展是至关重要的，因为单线程性能往往会达到一个平台。这种启用需要应用程序各个线程之间的有效通信，消除资源的不必要消耗以及其他多线程程序常见的问题。

值得一提的是，性能收益不仅仅来自调整软件。根据 [Leiserson et al., 2020]，未来的两个主要潜在速度提升源是算法（特别是对于机器学习等新问题领域）和简化的硬件设计。算法显然在应用程序的性能中发挥了重要作用，但我们不会在本书中讨论这个主题。我们也不会讨论新硬件设计的主题，因为大多数情况下，软件开发人员必须处理现有的硬件。然而，了解现代 CPU 设计对于优化应用程序是重要的。

> “在摩尔定律后的时代，使代码运行快速变得越来越重要，尤其是使其适合运行的硬件。” [Leiserson et al., 2020]

本书的方法论侧重于从应用程序中挤出最后一点性能。这种转变可以归因于表1中的第 6 行和第 7 行。将讨论的改进类型通常不大，并且通常不超过 10%。然而，不要低估 10% 的性能提升的重要性。这对于在云配置中运行的大型分布式应用程序尤为重要。根据 [Hennessy, 2018]，在 2018 年，谷歌在运行云的实际计算服务器上花费的资金大致与花费在电力和冷却基础设施上的资金相同。能源效率是一个非常重要的问题，可以通过优化软件来改善。

> “在这种规模下，理解性能特征变得至关重要 - 即使是性能或利用率的小幅改进也可以转化为巨大的成本节省。” [Kanev et al., 2015]

1.2 谁需要性能调优?

在像高性能计算（HPC）、云服务、高频交易（HFT）、游戏开发和其他对性能要求极高的领域，性能工程无需多加辩解。例如，谷歌曾报告说，搜索慢 2% 导致每位用户搜索减少 2%。[3] 对于 Yahoo! 来说，页面加载速度快了 400 毫秒导致流量增加了 5-9%。[4] 在这个数字游戏中，微小的改进可能产生显著影响。这些例子证明，服务工作得越慢，使用的人就越少。

有一句著名的话：“过早优化是万恶之源”。但相反的情况也经常成立。推迟性能工程工作可能为时已晚，导致的问题可能和过早优化一样恶劣。对于从事性能关键项目的开发人员来说，了解底层硬件工作原理是至关重要的。在这些行业中，如果一个程序在开发过程中没有以硬件为焦点，那就是注定会失败的。软件的性能特性必须与正确性和

[3] Marissa Mayer 的演讲幻灯片 - https://assets.en.oreilly.com/1/event/29/Keynote Presentation 2.pdf

[4] Stoyan Stefanov 的演讲幻灯片 - https://www.slideshare.net/stoyan/dont-make-me-wait-or-building-highperformance-web-applications

安全性一样受到重视，从第一天开始。ClickHouse 数据库就是一个成功的软件产品，它是围绕一个小而非常高效的核心构建的。

有趣的是，性能工程不仅在上述领域需要。如今，在通用应用程序和服务领域也需要性能工程。我们每天使用的许多工具，如果不能满足其性能要求，就根本不会存在。例如，集成到 Microsoft Visual Studio IDE 中的 Visual C++ IntelliSense 功能具有非常严格的性能约束。为了使 IntelliSense 自动完成功能正常工作，它们必须在毫秒级别内解析整个源代码库。[5] 如果源代码编辑器花费几秒钟才能建议自动完成选项，那么没有人会使用它。这样的功能必须非常响应迅速，并在用户输入新代码时提供有效的继续建议。只有通过注重性能并经过深思熟虑的性能工程，才能取得类似应用程序的成功。

有时，快速的工具会在它们最初设计的领域之外找到用途。例如，如今，游戏引擎如 Unreal 和 Unity 在建筑、3D 可视化、电影制作等领域中也得到了应用。由于游戏引擎非常高效，它们是需要 2D 和 3D 渲染、物理引擎、碰撞检测、声音、动画等功能的应用程序的自然选择。

> “快速的工具不仅允许用户更快地完成任务，还允许用户以全新的方式完成完全新的任务。” - Nelson Elhage 在他的博客文章中写道（2020 年）[6]。

我希望不用说人们讨厌使用慢软件。应用程序的性能特性可能是用户切换到竞争对手产品的唯一因素。通过强调性能，您可以为您的产品赢得竞争优势。

性能工程是一项重要而有益的工作，但可能非常耗时。事实上，性能优化是一场永无止境的游戏。总会有一些地方需要优化。不可避免地，开发人员会达到边际收益的点，在这一点上，进一步的改进将以非常高的工程成本为代价，并且可能不值得努力。应用程序针对硬件理论极限的性能评估有助于了解优化的潜在空间。知道何时停止优化是性能工作的关键方面。有些组织通过将这些信息集成到代码审查流程中来实现：源代码行用相应的“成本”指标进行注释。使用这些数据，开发人员可以决定是否值得优化代码的特定部分。

在开始性能调优之前，请确保有足够的理由这样做。为了优化而优化是没有价值的，如果它不能为您的产品增加价值。慎重的性能工程始于明确定义的性能目标，说明您试图实现什么以及为什么这样做。此外，您还应该选择用于衡量是否达到目标的指标。您可以在 [Gregg, 2013] 和 [Akinshin, 2019] 中阅读有关设置性能目标的更多信息。

尽管如此，实践和掌握性能分析和调优的技能总是很好的。如果您因此原因拿起了这本书，那么非常欢迎您继续阅读。

1.3 什么是性能分析?

曾经和同事辩论过某段代码的性能吗？那么你可能知道要预测哪段代码会表现最佳有多么困难。在现代处理器内部有如此多的运作部件，即使对代码进行微小的调整也可能触发显著的性能变化。这就是为什么本书的第一个建议是：**始终进行测试**。许多人在尝试优化他们的应用程序时依赖直觉。而通常情况下，最终会在各处随意修复，而对应用程序的性能没有产生任何真正的影响。

经验不足的开发人员经常在源代码中进行更改，并希望它会改善程序的性能。一个这样的例子是在代码库中到处用 `i++` 替换为 `++i`，假设之前的 `i` 值未被使用。在一般情况下，这种更改对生成的代码不会有任何影响，因为任何像样的优化编译器都会认识到之前的 `i` 值未被使用，并会消除多余的副本。

许多围绕世界流传的微优化技巧在过去是有效的，但现在的编译器已经学会了它们。此外，一些人倾向于过度使用传统的位操作技巧。其中一个例子是使用XOR 交换算法[7]，而事实上，简单的`std::swap`会产生更快的代码。这种意外的更改可能不会改善应用程序的性能。找到正确的修复位置应该是经过仔细的性能分析得出的结果，而不是靠直觉和猜测。

[5] 事实上，不可能在毫秒级的时间内解析整个代码库。相反，IntelliSense 只重构 AST 中已更改的部分。请在视频中观看微软团队如何实现这一目标的更多细节: https://channel9.msdn.com/Blogs/Seth-Juarez/Anders-Hejlsberg-on-Modern-Compiler-Construction.

[6] N. Elhage 的软件性能反思 - https://blog.nelhage.com/post/reflections-on-performance/

[7] xor 交换算法 - https://en.wikipedia.org/wiki/XOR_swap_algorithm

有许多性能分析方法可能会或可能不会带领你发现问题。本书介绍的针对特定 CPU 的性能分析方法有一个共同点:它们都基于收集有关程序执行方式的特定信息。在程序源代码中做出的任何更改都是通过分析和解释收集的数据来驱动的。

找到性能瓶颈只是工程师工作的一半。第二半部分是正确地修复它。有时,在程序源代码中更改一行代码可能会带来显著的性能提升。性能分析和调优就是要找到并修复这行代码。错过这样的机会可能会是一种巨大的浪费。

1.4 本书讨论了什么?

本书旨在帮助开发人员更好地理解其应用程序的性能,学会发现效率低下的地方,并加以消除。为什么我的手写压缩算法比传统的慢了两倍?为什么我的函数更改导致性能下降了一半?客户抱怨我的应用程序运行缓慢,我应该从哪里开始?我是否已经充分优化了程序?我的平台上有哪些性能分析工具?减少缓存未命中和分支预测失败的技术有哪些?希望通过本书的学习,你能找到这些问题的答案。

以下是本书的大纲:

- 第二章讨论如何进行公平的性能实验并分析其结果。它介绍了性能测试和比较结果的最佳实践。
- 第三章介绍了 CPU 微体系结构的基础知识,第四章介绍了性能分析中使用的术语和指标;我们建议即使你认为你已经了解这些内容,也不要跳过这些章节。
- 第五章探讨了几种最流行的性能分析方法。它解释了分析技术的工作原理,可以收集哪些运行时数据以及如何进行收集。
- 第六章审查了现代 CPU 提供的功能,以支持和增强性能分析。它展示了它们的工作原理以及它们可以解决的问题。
- 第七章概述了主要平台上最受欢迎的工具,包括基于 x86 和 ARM 的处理器上运行的 Linux、Windows 和 MacOS。
- 第八至十一章包含了典型性能问题的解决方案。这些章节根据自顶向下微体系结构分析方法进行组织,这是本书最重要的概念之一。如果一些术语对您还不清楚,不要担心,我们将一步一步地覆盖所有内容。第八章(内存瓶颈 (Memory Bound))涉及优化内存访问、缓存友好的代码、内存分析、大页和其他一些技术。第九章(核心绑定)涉及优化计算,探讨了函数内联、循环优化和向量化。第十章(坏预测)涉及无分支编程,用于避免频繁的预测失败分支。第十一章(前端瓶颈)涉及机器代码布局优化,如基本块放置、函数拆分、基于优化分析文件的优化等。
- 第十三章包含了不特定于前四章所涵盖的任何类别的优化主题,但仍然足够重要,以至于在本书中找到了它们的位置。在这里,你会找到低延迟技术、调整系统以获得最佳性能的技巧、标准库函数的更快替代方法等。
- 第十四章讨论了分析多线程应用程序的技术。它概述了优化多线程应用程序性能的一些最重要挑战以及可用于分析它们的工具。这个主题本身很庞大,所以这一章节只侧重于硬件相关的问题,比如“伪共享”。
- 第十五章讨论了软件和硬件性能领域的当前和未来趋势。我们讨论了传统计算机系统设计的进步以及一些创新的想法。

本书提供的示例主要基于开源软件:Linux 作为操作系统,基于 LLVM 的 Clang 编译器用于 C 和 C++ 语言,以及 Linux `perf`作为性能分析工具。选择这些技术的原因不仅是因为它们的流行程度,还因为它们的源代码是开放的,这使我们能够更好地理解它们的工作原理。这对于学习本书中提出的概念特别有用。我们有时也会展示一些专有工具,这些工具在其领域是“大玩家”,例如 Intel® VTune™ Profiler。

1.5 本书未涉及的内容

系统性能取决于不同的组件:CPU、操作系统、内存、I/O 设备等。应用程序可以通过调整系统的各个组件来获益。一般来说,工程师应该分析整个系统的性能。然而,系统性能的最大因素是其核心,即 CPU。这就是为什么本书主要从 CPU 的角度进行性能分析,偶尔涉及操作系统和内存子系统。

本书的范围不超出单个 CPU 插槽，因此我们不会讨论分布式、NUMA 和异构系统的优化技术。使用诸如 OpenCL 和 openMP 之类的解决方案将计算外包给加速器（GPU、FPGA 等）在本书中也没有讨论。

本书以 Intel x86-64 CPU 架构为中心，并不提供针对 AMD、ARM 或 RISC-V 芯片的具体调优方案。尽管如此，本书讨论的许多原则也适用于这些处理器。此外，Linux 是本书选择的操作系统，但对于我们提供的大多数示例来说，这并不重要，因为相同的技术也有益于运行在 Windows 和 macOS 操作系统上的应用程序。

本书中的所有代码片段均以 C、C++ 或 x86 汇编语言编写，但在很大程度上，本书中的思想也适用于其他编译为本机代码的语言，如 Rust、Go，甚至 Fortran。由于本书针对的是靠近硬件运行的用户模式应用程序，因此我们不会讨论托管环境，例如 Java。

最后，作者假设读者对他们开发的软件拥有完全控制权，包括他们使用的库和编译器的选择。因此，本书不涉及调整购买的商业软件包，例如调整 SQL 数据库查询。

1.6 练习

作为本书的补充材料，我们开发了一系列免费的实验任务，可以在https://github.com/dendibakh/perf-ninja上获取。性能忍者是一个在线课程，您可以在其中练习低级性能分析和调优。我们在整本书中都提供了该仓库中的实验任务。例如，当您看到`perf-ninja::warmup`时，这对应于位于上述仓库的`labs/misc/warmup`文件夹中的实验任务。

您可以在本地计算机上解决这些任务，也可以将您的代码提交到 Github 进行自动化基准测试。如果选择后者，请按照仓库的“入门”页面上的说明操作。如果您遇到问题，可以查看与实验任务相关的视频。这些视频解释了问题的可能解决方案。

章节总结

- CPU 的单线程性能增长速度不如几十年前那样迅猛。这就是为什么性能调优变得比过去 40 年更加重要的原因。计算行业正在变化，现在比 90 年代任何时候都更加剧烈。
- 根据 [Leiserson et al., 2020] 的观点，未来软件调优将成为性能提升的关键驱动因素之一。性能调优的重要性不容小觑。对于大型分布式应用程序，每一点性能改进都会带来巨大的成本节约。
- 软件默认情况下并不具有最佳性能。存在某些限制，阻止应用程序发挥其全部性能潜力。HW 和 SW 环境都有这样的限制。CPU 不能神奇地加速缓慢的算法。编译器远不能为每个程序生成最佳代码。由于硬件的特定性，某个特定问题的最佳算法并不总是性能最佳的。所有这些都为我们调优应用程序的性能留下了空间。
- 对于某些类型的应用程序，性能不仅仅是一个特性。它使用户能够以新的方式解决新的问题。
- 软件优化应该有强有力的业务需求支持。开发人员应该设定可量化的目标和度量标准，这些目标和度量标准必须用于衡量进展。
- 预测某段代码的性能几乎是不可能的，因为现代平台有很多因素影响性能。在实施软件优化时，开发人员不应依赖直觉，而是应该使用谨慎的性能分析。